Module Interface Specification for Software Engineering

Team 11, technically functional
Matthew Huynh
Cieran Diebolt
Vaisnavi Shanthamoorthy
Maham Siddiqui
Eman Ashraf

November 7, 2025

1 Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

2 Symbols, Abbreviations and Acronyms

See SRS Documentation at [give url —SS]

[Also add any additional symbols, abbreviations or acronyms —SS]

Contents

3 Introduction

The following document details the Module Interface Specifications for [Fill in your project name and description —SS]

Complementary documents include the System Requirement Specifications and Module Guide. The full documentation and implementation can be found at .... [provide the url for your repo —SS]

4 Notation

[You should describe your notation. You can use what is below as a starting point. —SS]

The structure of the MIS for modules comes from Hoffman and Strooper (1995), with the addition that template modules have been adapted from Ghezzi et al. (2003). The mathematical notation comes from Chapter 3 of Hoffman and Strooper (1995). For instance, the symbol := is used for a multiple assignment statement and conditional rules follow the form $(c_1 \Rightarrow r_1 | c_2 \Rightarrow r_2 | ... | c_n \Rightarrow r_n)$.

The following table summarizes the primitive data types used by Software Engineering.

Data Type	**Notation**	**Description**
character	char	a single symbol or digit
integer	$\mathbb{Z}$	a number without a fractional component in (-∞, ∞)
natural number	$\mathbb{N}$	a number without a fractional component in [1, ∞)
real	$\mathbb{R}$	any number in (-∞, ∞)

The specification of Software Engineering uses some derived data types: sequences, strings, and tuples. Sequences are lists filled with elements of the same data type. Strings are sequences of characters. Tuples contain a list of values, potentially of different types. In addition, Software Engineering uses functions, which are defined by the data types of their inputs and outputs. Local functions are described by giving their type signature followed by their specification.

5 Module Decomposition

The following table is taken directly from the Module Guide document for this project.

Level 1	Level 2
Hardware-Hiding	
Behaviour-Hiding	Input Parameters Output Format Output Verification Temperature ODEs Energy Equations Control Module Specification Parameters Module
Software Decision	Sequence Data Structure ODE Solver Plotting

Table 1: Module Hierarchy

6 Variables

Name	Type	Description
Name	String	This is the name of the user.
role	String	Values can be PT or patient
email	String	This will be used as their login ID
password	String	This is set by the user for login purposes
birthday	String	This is the user's birthday that will be used for verification purposes
license_no	Int	This is used for verification of PT
invite_code	String	This is given by the PT to the patient to allow registration on the application
Recent_report	ADT	Generated report
Date	String	The date selected by user
Recent_analysis	ADT	Most recently generated analysis
Report	Record	This is the report(s) being sent to the dashboard for analysis.

Table 2: User Account Data Fields

7 MIS

7.1 Module - Account Management

ADT

This module is an adapter that connects to the user database. This database consists of these tables:

1. User data: this consists of information such as name, role, email, password, birthday, and license_no/invite_code.

2. Exercise data: this table consists of exercise nstructions, maximum height (the highest point their leg can lift up to), minimum height (starting point) and UserAccount_P['Name'].

3. User activity: This will contain information about a specific patient's exercise activity. Data such as exercise duration, maximum height, number of repetitions, target area, the date the exercise was performed and its time, rest time, number of sets and analysis. Additionally, it will include a column for the patient's feedback on their exercise performance.

7.1.1 Uses

- name
- role
- email
- password
- birthday
- license_no
- invite_code
- exercise
- time
- date
- duration

7.1.2 Syntax

Exported Constants

- *UserAccount_P*: ADT
- *UserAccount_PT*: ADT

Exported Access Programs

Access Routine Name	Input	Output	Exceptions
get_userdb_info	*NA*	UserAccount_P OR UserAccount_PT	User_not_found Download_error
PTaccount_delete	*Name* *email*	NA	required_field_empty patient_not_found
username_pw_match	*Email* *Password*	NA	wrong_email_password
add_analysis_to_user_act	*UserAccount_P* *Recent_analysis*	NA	User_not_found Upload_error

Access Routine Name	Input	Output	Exceptions
save_video	*UserAccount_P* *Recent_video*	NA	User_not_found Upload_error Video_too_short
get_analysis	*UserAccount_P* *Date*	Recent_video	User_not_found Download_error
exerc_instruct	*UserAccount_P* *Exercise*	*NA*	User_not_found Download_error

7.1.3 Semantics

State Variables

A: an instance of UserAccount
B: an instance of UserAccount

State Invariant

$\forall A, B \in UserDatabase : A[i] \neq B[j]$
$0 < A < length(A) \cap A[i] \notin \emptyset$

7.1.4 Assumptions

7.1.5 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

7.1.6 Local Functions

Access Routine Name	Input	Output	Exceptions
account_create	*Name* *Role* *Birthday* *License_no/ Invite_code* *Email* *Password*	UserAccount_P or UserAccount_PT	Input_format_invalid Required_field_empty Account_already_exists Incorrect_creds Invite_code_expired
set_user_info	*Name* *Role* *Birthday* *License_no/ Invite_code* *Email* *Password*	UserAccount_P or UserAccount_PT	Input_format_invalid Required_field_empty Account_already_exists Incorrect_creds Invite_code_expired
send_db_analysis	*UserAccount_P*	Recent_analysis	User_not_found Download_error
send_db_video	*UserAccount_P*	Recent_video	User_not_found Download_error
set_analysis_report	*UserAccount_P*	Recent_analysis	User_not_found Download_error

Table 4: Account Management Access Routines

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

7.2 Module - Dashboard

ADT

This module's purpose is to display the most recent report, an overall analysis of progress, and rehabilitation insights.

7.2.1 Uses

- Recent_report
- Report_db
- UserAccount_P

7.2.2 Syntax

Exported Constants

- *UserAccount_Patient*: ADT
- *UserAccount_PT*: ADT

Exported Access Programs

7.2.3 Semantics

State Variables

A: an instance of UserAccount
B: an instance of UserAccount

State Invariant

$\forall A, B \in UserDatabase : A[i] \neq B[j]$
$0 < A < length(A) \cap A[i] \notin \emptyset$

7.2.4 Assumptions

7.2.5 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

7.3 Module - Analyze

NA

7.3.1 Uses

7.3.2 Syntax

Exported Constants
Exported Access Programs

Access Routine Name	Input	Output	Exceptions
placeholder	*NA*	UserAccount_P OR UserAccount_PT	User_not_found Download_error

7.3.3 Semantics

State Variables

State Invariant

7.3.4 Assumptions

7.3.5 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

7.3.6 Local Functions

Access Routine Name	Input	Output	Exceptions
placeholder	*Name* *Role* *Birthday* *License_no/* *Invite_code* *Email* *Password*	UserAccount_P or UserAccount_PT	Input_format_invalid Required_field_empty Account_already_ exists Incorrect_creds Invite_code_expired

Table 6: Account Management Access Routines

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

7.3.7 Local Functions

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

References

Carlo Ghezzi, Mehdi Jazayeri, and Dino Mandrioli. *Fundamentals of Software Engineering*. Prentice Hall, Upper Saddle River, NJ, USA, 2nd edition, 2003.

Daniel M. Hoffman and Paul A. Strooper. *Software Design, Automated Testing, and Maintenance: A Practical Approach*. International Thomson Computer Press, New York, NY, USA, 1995. URL http://citeseer.ist.psu.edu/428727.html.

8 Appendix

[Extra information if required —SS]

Appendix — Reflection

[Not required for CAS 741 projects —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Problem Analysis and Design.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. Which of your design decisions stemmed from speaking to your client(s) or a proxy (e.g. your peers, stakeholders, potential users)? For those that were not, why, and where did they come from?
4. While creating the design doc, what parts of your other documents (e.g. requirements, hazard analysis, etc), it any, needed to be changed, and why?
5. What are the limitations of your solution? Put another way, given unlimited resources, what could you do to make the project better? (LO_ProbSolutions)
6. Give a brief overview of other design solutions you considered. What are the benefits and tradeoffs of those other designs compared with the chosen design? From all the potential options, why did you select the documented design? (LO_Explores)